print("Hello, World!")

LINGUAGEM DE PROGRAMAÇÃO PYTHON

AULA 07

SANTA CRUZ
CENTRO UNIVERSITÁRIO

ANTERIORMENTE

- **Estruturas de dados nativas do Python;**
- **Listas, tuplas, dicionários, conjuntos;**
- **Métodos mais comuns;**
- **Exercícios.**

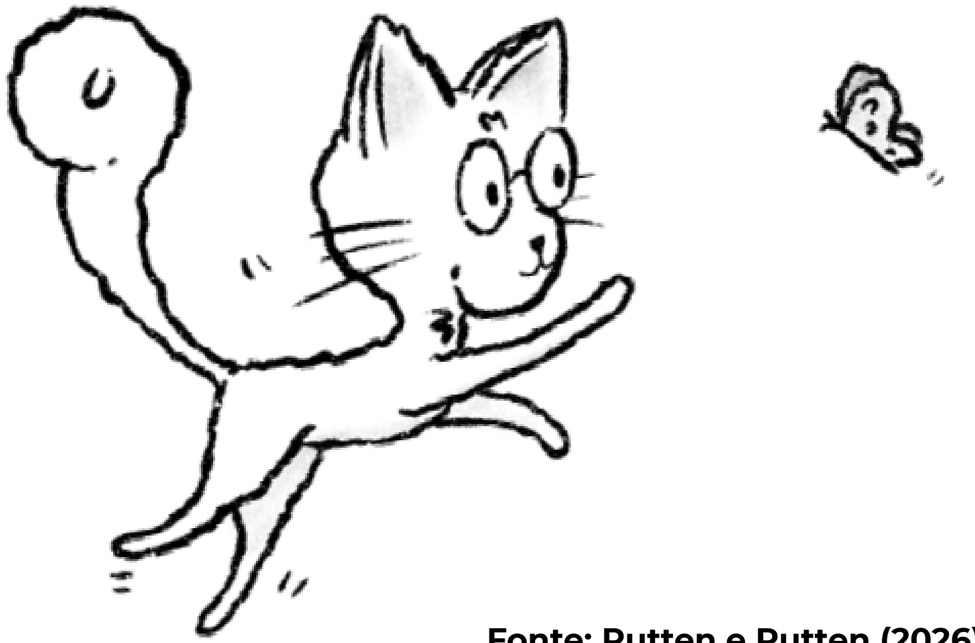
SANTA CRUZ
CENTRO UNIVERSITÁRIO

PRÓXIMOS PASSOS

- Compreender o conceito de debugging e sua importância.
- Identificar e diferenciar os três principais tipos de bugs.
- Aprender a decifrar mensagens de erro (Tracebacks).
- Utilizar ferramentas essenciais.
- Aplicar um fluxo de trabalho para resolver problemas.
- Try - Except

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Caçando “bugs”



Fonte: Putten e Putten (2026)

O que é um Bug?

- **Origem do termo:** Falhas ou comportamentos inesperados em um programa de computador.
- **Definição de Debugging:** O processo sistemático de identificar, analisar e corrigir esses problemas no código.
- **A Realidade do Desenvolvimento:** Escrever código é apenas metade do caminho. A outra metade é garantir que ele funcione conforme o esperado.

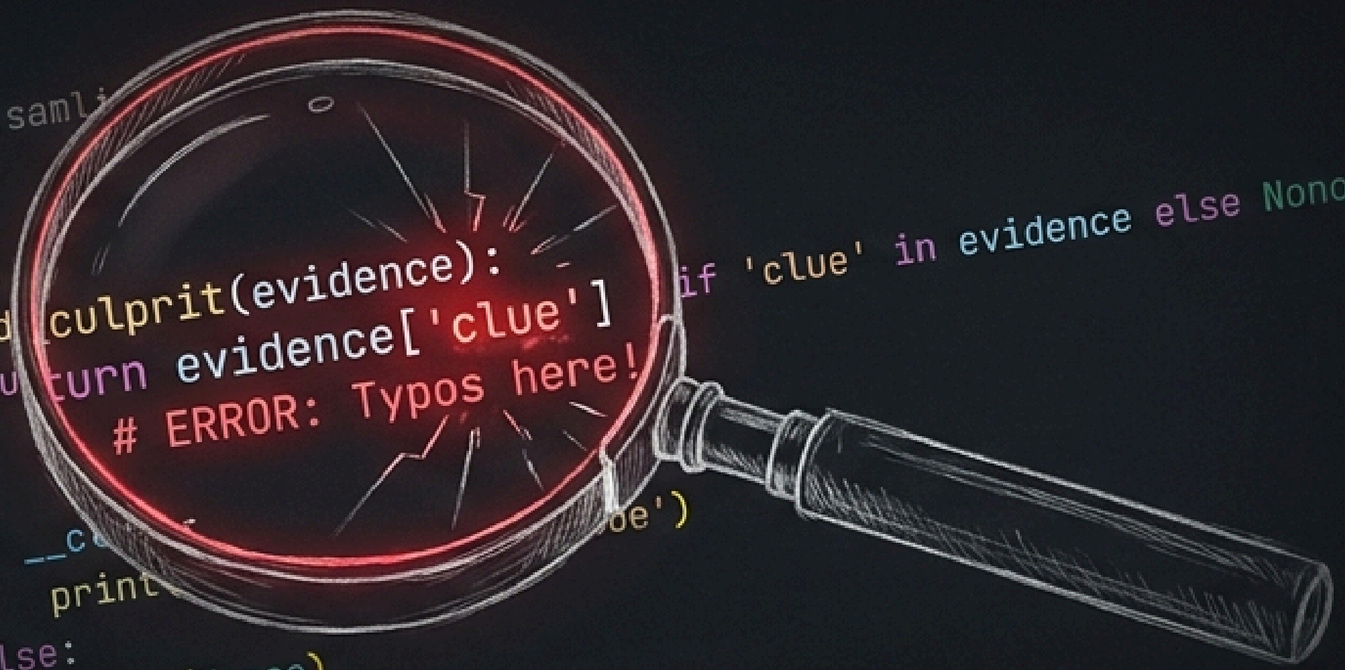
Fonte: Putten e Putten (2026)

Por Que Aceitar os Erros?

- **Parte do aprendizado:** Erros não significam que você é um mau programador. Até profissionais sêniores lidam com bugs diariamente.
- **Desenvolvimento de habilidades:** Quanto mais você depura, mais rápida se torna sua capacidade de leitura e compreensão de sistemas complexos.

Fonte: Putten e Putten (2026)

SANTA CRUZ
CENTRO UNIVERSITÁRIO



A Arte do Debugging

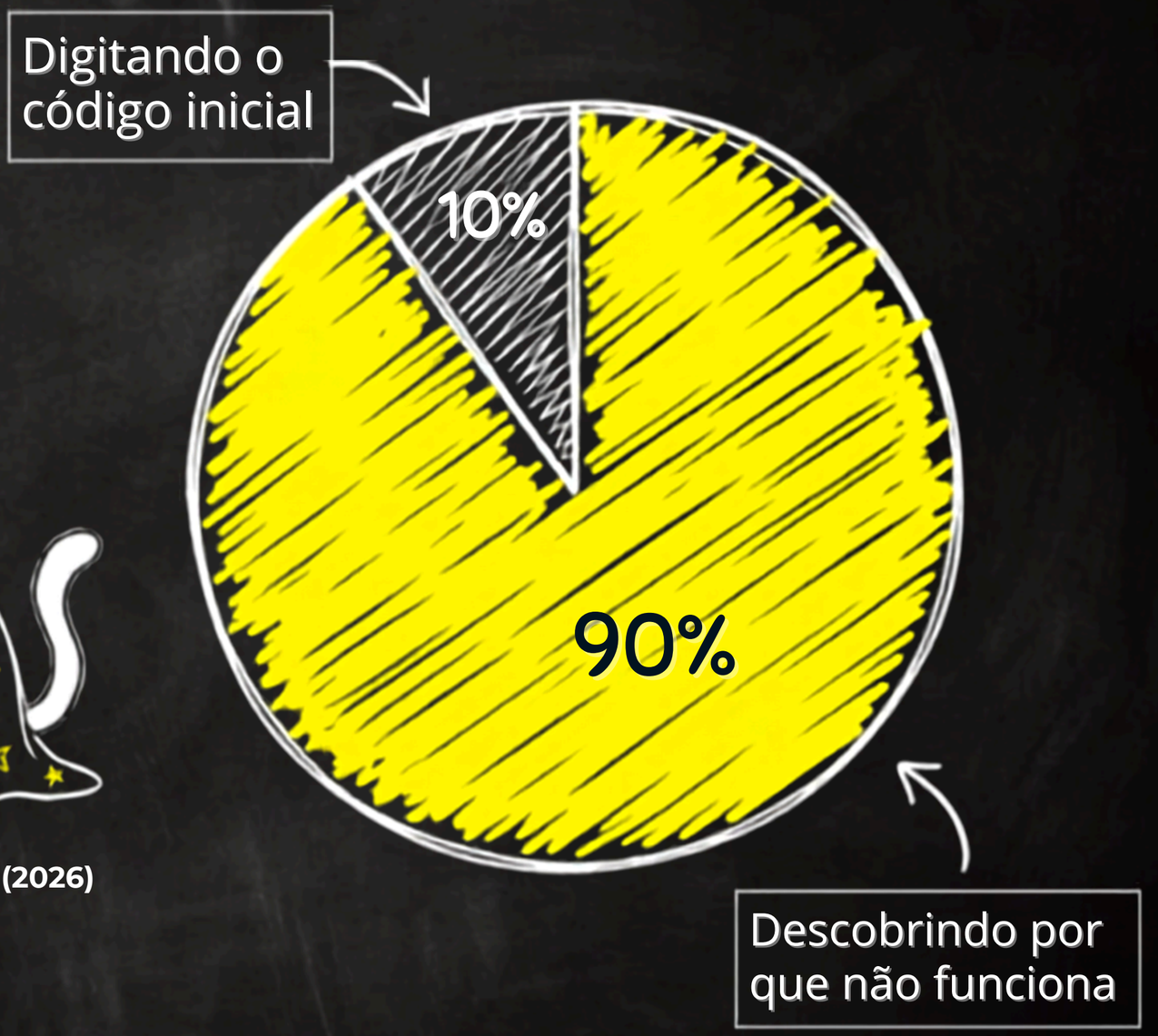
Como solucionar mistérios e investigar a cena do crime no seu código Python.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

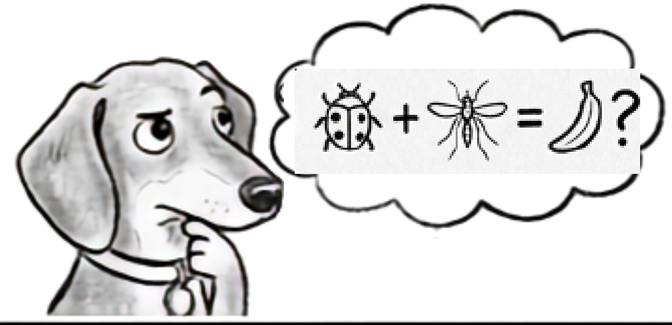
Todo Código Esconde Segredos

Fonte: Putten e Putten (2026)

A Realidade do Desenvolvedor



Os Suspeitos



O Óbvio	O Inesperado	O Silencioso
Erro de Sintaxe (Syntax Error)	**Erro de Tempo de Execução (Runtime Error)**	**Erro Lógico (Logical Error)**
O código nem roda. Regras gramaticais quebradas (ex: falta de :, aspas abertas).	*Ação impossível (ex: divisão por zero, acesso a índice inexistente).*	*O código roda sem reclamar, mas o resultado é bizarro ou incorreto. O mais perigoso e furtivo dos três.*

Gemini Notebook

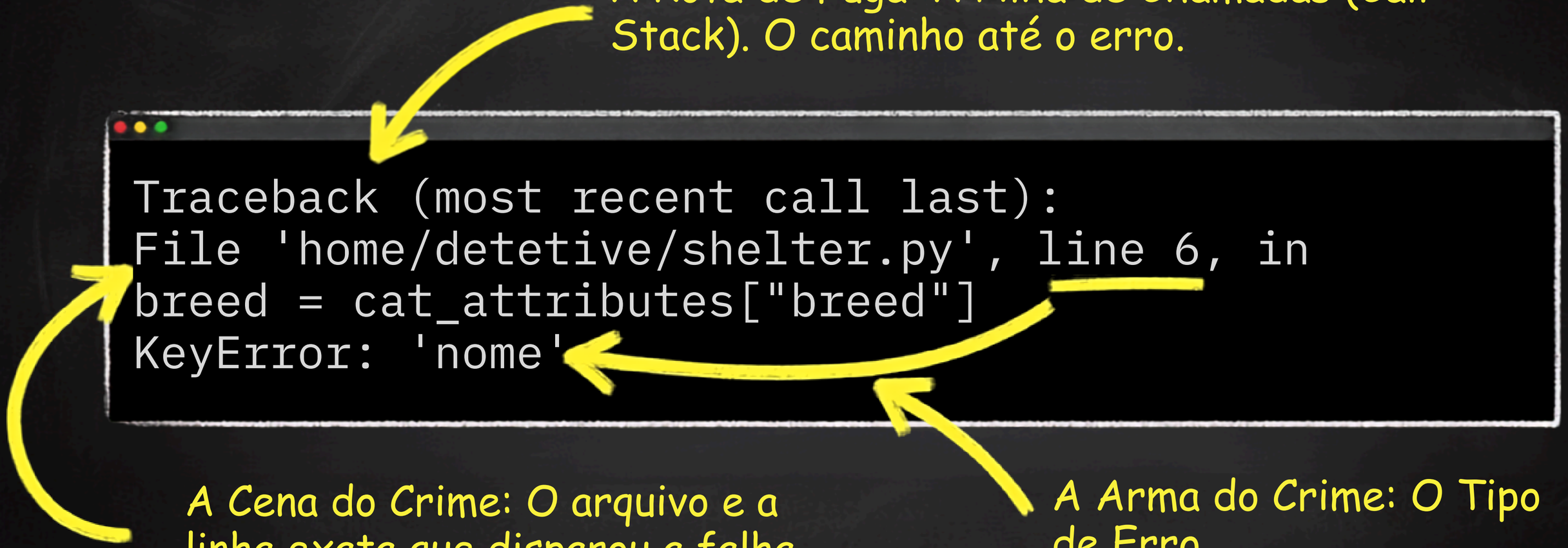
Entendendo mensagens de erro

```
cat_attributes = {
    "name": "Zia",
    "age": 1,
    "color": "grey",
    }
breed = cat_attributes["breed"]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipykernel_898/512590257.py in <cell line: 0>()
      4     "color": "grey",
      5     }
----> 6 breed = cat_attributes["breed"]

KeyError: 'breed'
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Lendo os Rastros: A Anatomia de um Traceback



Matriz de Diagnóstico: Inimigos Comuns

Tipo de Erro	O que significa	Exemplo Prático
NameError	Variável fantasma	Tentar usar uma variável nunca definida.
TypeError	Mistura incompatível	Somar texto com número.
ValueError	Tipo certo, conteúdo errado	Tentar transformar a letra 'A' em número inteiro.
IndexError	Fora dos limites	Acessar a posição 10 em uma lista de 3 itens.
KeyError	Chave inexistente	Procurar um termo que não existe no dicionário.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 01 Aula 07

Caça aos Bugs (Traceback)

Execute os códigos abaixo, um por vez. O programa vai 'quebrar' e exibir um erro no terminal.

Objetivo: Ler o traceback, descobrir qual é a linha do erro, qual é o tipo do erro e corrigir o código para que ele funcione, adicione um comentário sobre a correção.

```
nome_heroi = "Batman"
print("O herói que vai salvar a cidade é o " + nome heroi cidade)
```

```
idade_pet = 5
mensagem = "A idade do meu cachorro é " + idade_pet + " anos."
print(mensagem)
```

```
print("Calculadora de Dobro!")
# Simulando alguém que digitou a palavra 'dez' em vez do número 10
numero_digitado = "dez"
dobro = int(numero_digitado) * 2
print("O dobro é:", dobro)
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 01 Aula 07

```
cat_attributes = {
    "name": "Zia",
    "age": 1,
    "color": "grey"
}

# Tentando descobrir a raça (breed) da gata
raca_gato = cat_attributes["breed"]
print("A raça da gata é:", raca_gato)

some_numbers = [10, 20, 30]

# Tentando acessar o quarto número da lista
quarto_numero = some_numbers[3]
print("O número escolhido foi:", quarto_numero)
```

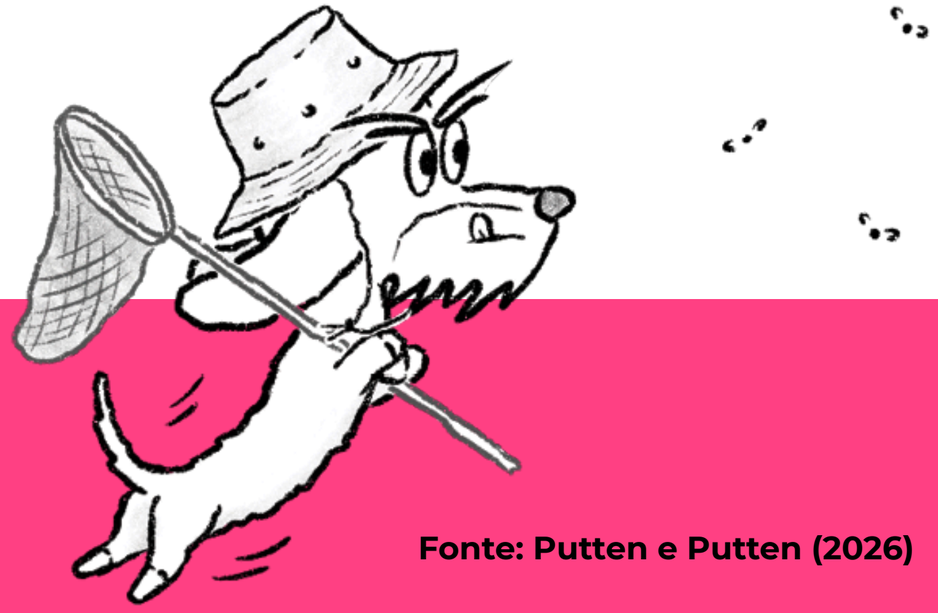
SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 01 Aula 07

```
tesouro_moedas = 100
aventureiros_sobreviventes = 0

print("Calculando a divisão do tesouro...")

moedas_por_pessoa = tesouro_moedas / aventureiros_sobreviventes

print("Cada aventureiro vai receber:", moedas_por_pessoa)
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Técnicas para encontrar bugs



Fonte: Putten e Putten (2026)

A Tática de Guerrilha: Iluminando Variáveis com print()

O **print()** funciona como uma lanterna em uma sala escura, revelando o estado invisível dos dados.

O Problema Escuro

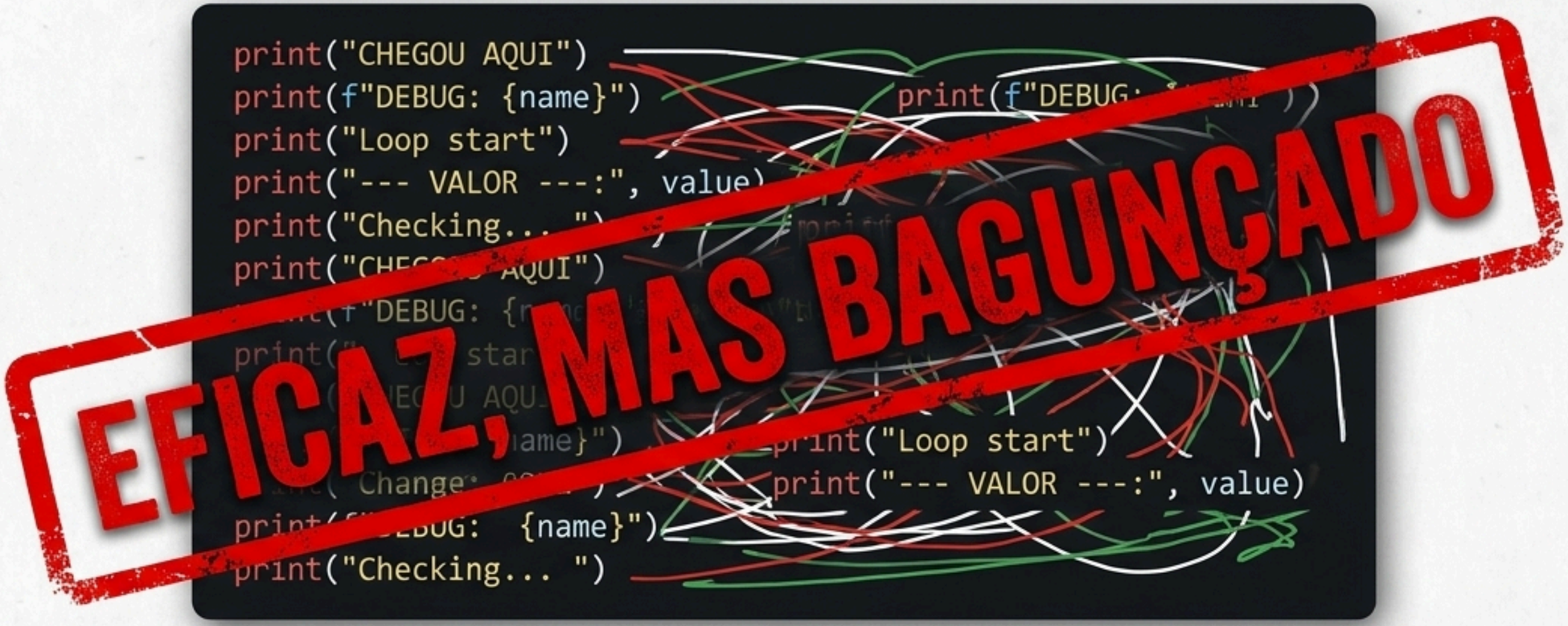
```
def feed_pet(name):
    pats on == null
    if petɪ[''nome'] == name:
        return 'fo' = {name}

    if name:
        pets = pet['nome']
        print("pet_pet.arrext = {name")
        print('nome' = name)
    else:
        return 'nome'
```

A Solução Iluminada

```
def feed_pet(name):
    pets on == null
    if pet[['nome'] == name:
        return 'fo' = {name}
    print(f"DEBUG: Verificando pet {name}")
    if name:
        pets = pet['name']
        print("pet_pet.arext = {name")
        print('nome' = name)
    else:
        return 'nome'
```

ATENÇÃO: Limpe a cena do crime! Remova todos os prints assim que o bug for resolvido.

A Lanterna no Escuro: Inspeção Manual com print()



A Vantagem

Rápido de inserir, funciona em qualquer ambiente.

O Problema

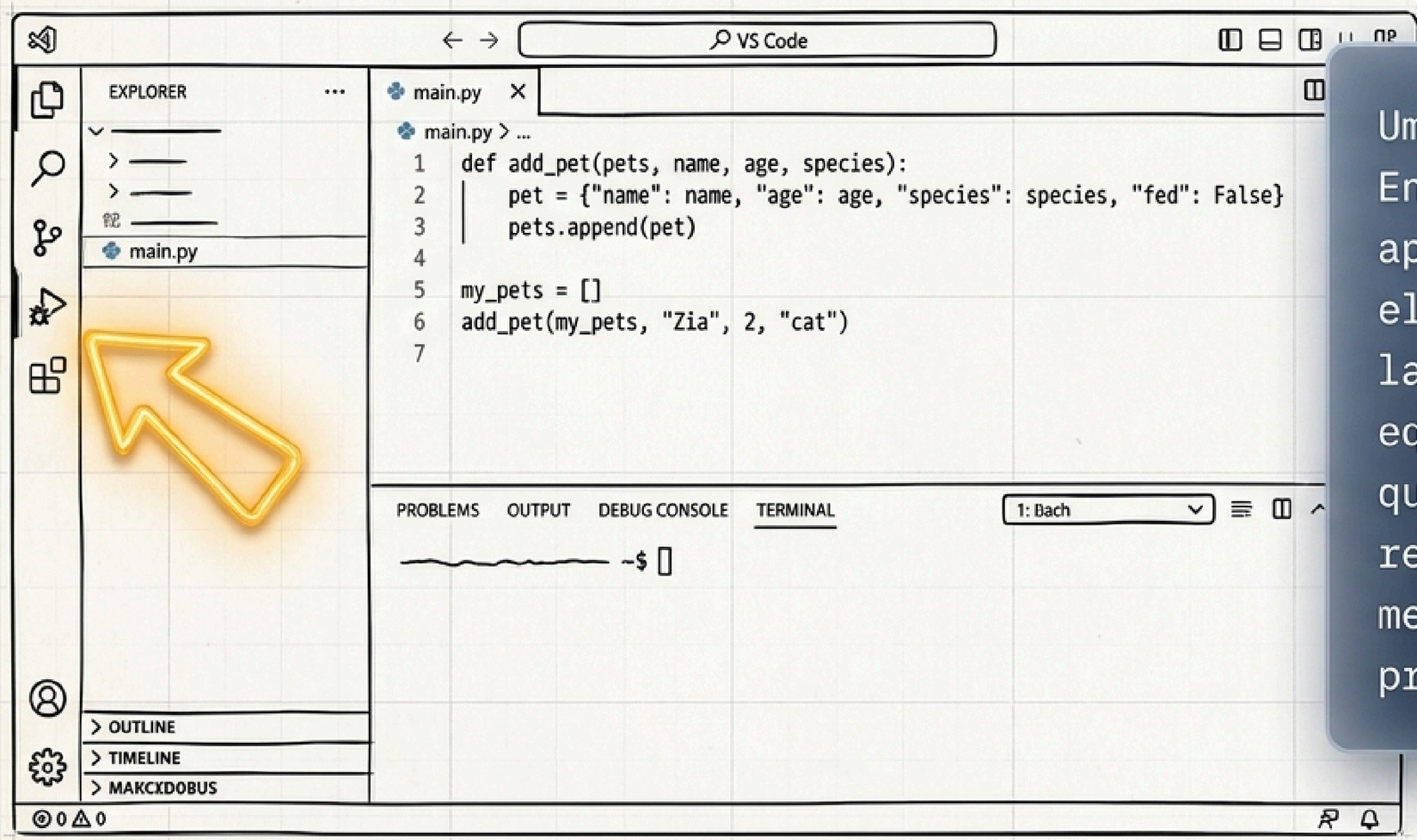
Como iluminar um galpão escuro com uma lanterna de bolso — você só vê uma pequena variável por vez. Em funções com muitos loops, sua tela é inundada de lixo visual.

O Veredito

Útil para checagens rápidas, ineficaz para arquiteturas e mistérios complexos.

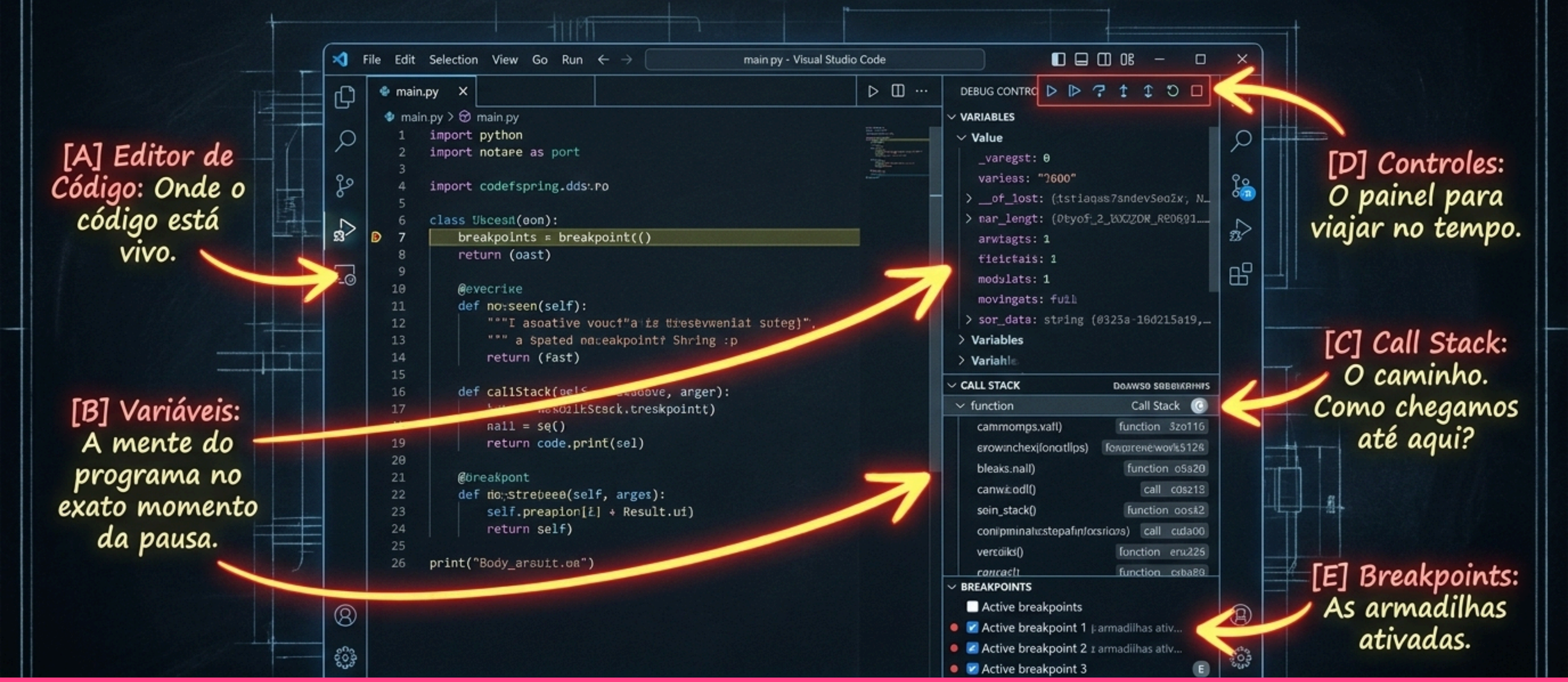
O Laboratório Forense (Entrando no IDE)

Abandone a lanterna e acenda as luzes do teto.



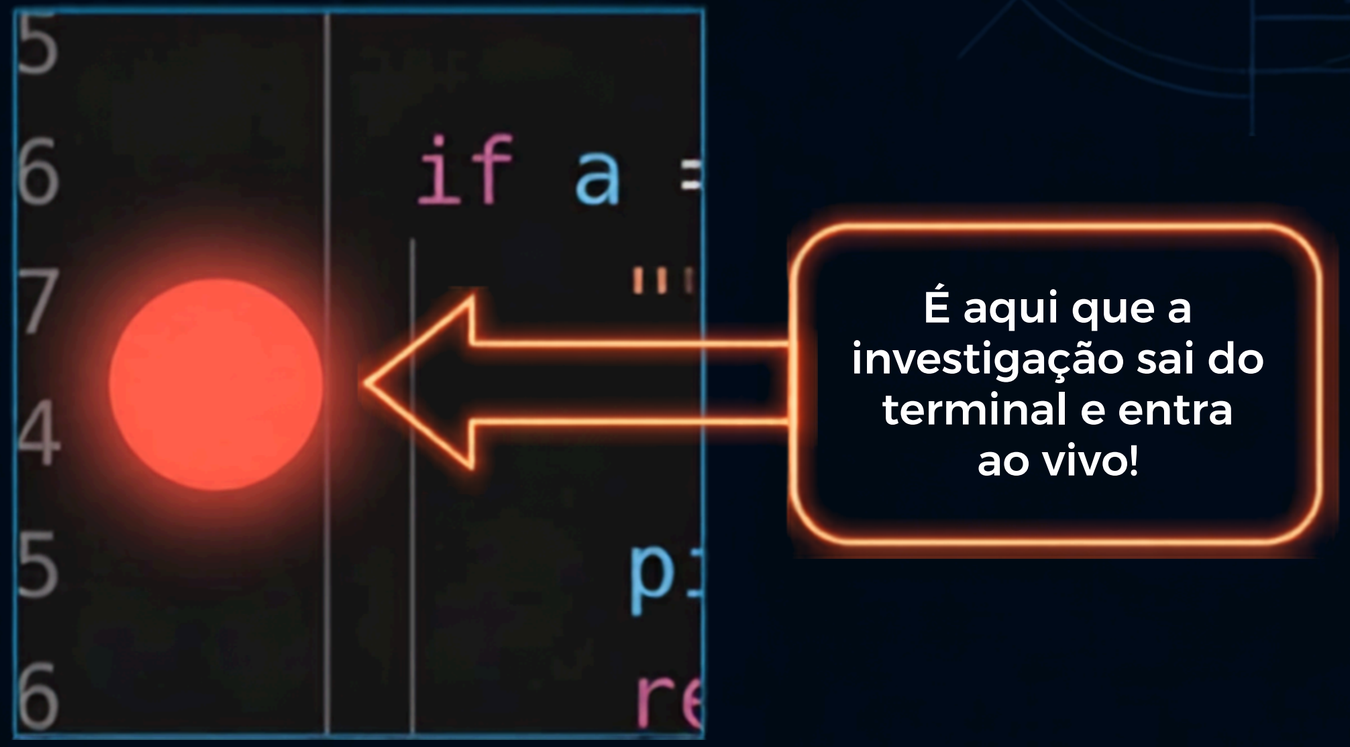
Um Integrated Development Environment (IDE) não é apenas um editor de texto elegante. É o seu laboratório forense equipado com ferramentas que permitem pausar a realidade e inspecionar os menores detalhes do seu programa.

O Laboratório Forense: Mapeando o Depurador da IDE



O Ponto de Interceptação: Invocando o Breakpoint

O programa executa, mas pausa exatamente na linha marcada com o sinal vermelho, antes de executá-la.



SANTA CRUZ
CENTRO UNIVERSITÁRIO

Congelando o Tempo: Movimentos Táticos

Continue

Siga em frente. O programa volta a rodar até o próximo ponto.

Step Over

Dê um passo à frente. Executa a linha inteira e pausa na próxima.

Step Into

Entrar. Entra dentro da função função para analisar os detalhes.

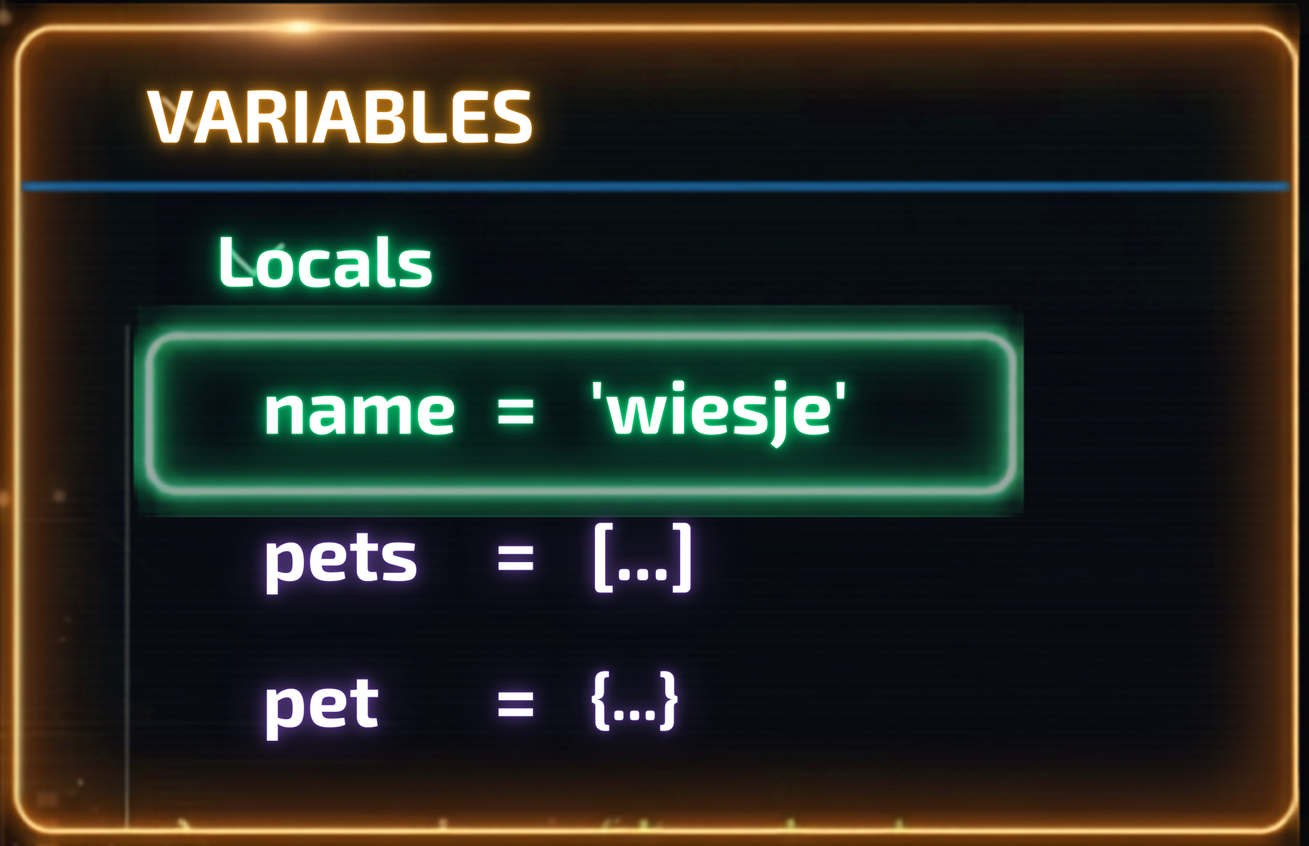
Step Out

Superfície. Termina a função atual e volta para quem a chamou.

Restart / Stop

Aborte a missão ou reinicie do zero.

O Raio-X da Cena: Inspetor de Variáveis



Anotações do Detetive:

- Locais: Variáveis vivas apenas na função atual.
- Globais: O contexto amplo do seu programa.
- Com o inspetor, todas as evidências estão visíveis simultaneamente. Se 2 + 2 = 5, este painel te mostrará exatamente onde o valor foi corrompido.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Seguindo as Pegadas (O Call Stack)

Como viemos parar aqui? O Call Stack é o ‘Diário de Bordo’ do seu código.

Se o seu código parou na linha 6 da função feed_pet, a pilha revela exatamente quem ordenou que essa função fosse chamada (ex: linha 18 do arquivo principal).

Crucial para rastrear a origem do “crime” em arquiteturas profundas com múltiplos módulos.

Direto no Terminal: O Módulo pdb

Para os detetives puristas trabalhando em servidores remotos sem interface gráfica.

```
def feed_pet(pets, name):
    breakpoint()   # <— O fluxo pausa magicamente aqui
    for pet in pets:
        print("Feeding...")
```

Funciona como a IDE, mas a navegação (n para next, c para continue) e inspeção de variáveis ocorrem inteiramente por linha de comando.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Módulo pdb

```
    def feed_pet(set):
        for pet_set_trace(5):
            print(Feeding the pet.,v);

        return feed  breakpoint()!
>>>
>>>     import pdb; pdb.set_trace()
>>>
```

Sem Interface?

E se você estiver em um servidor Linux sem interface gráfica (VS Code) para te ajudar?

A Solução

O Python tem o módulo pdb embutido.Basta digitar a palavra mágica no código: **breakpoint()**

Comandas do Terminal

n (next/step over), s (step into), c (continue).

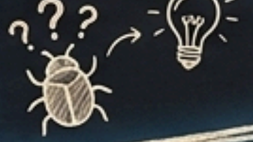
Aviso Legal

É uma arma essencial em níveis avançados.

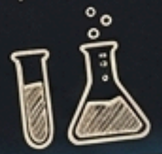
O Método Científico do Debugging
(A Rota do Investigador)

1. Reproduzir o Bug

2. Hipotetizar a Causa



3. Testar a Hipótese



4. Limpar a Cena

5. Verificar: Foi resolvido?

Não -> Tente novamente

B

A

Sim -> SUCESSO! ✓

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Gemini Notebook

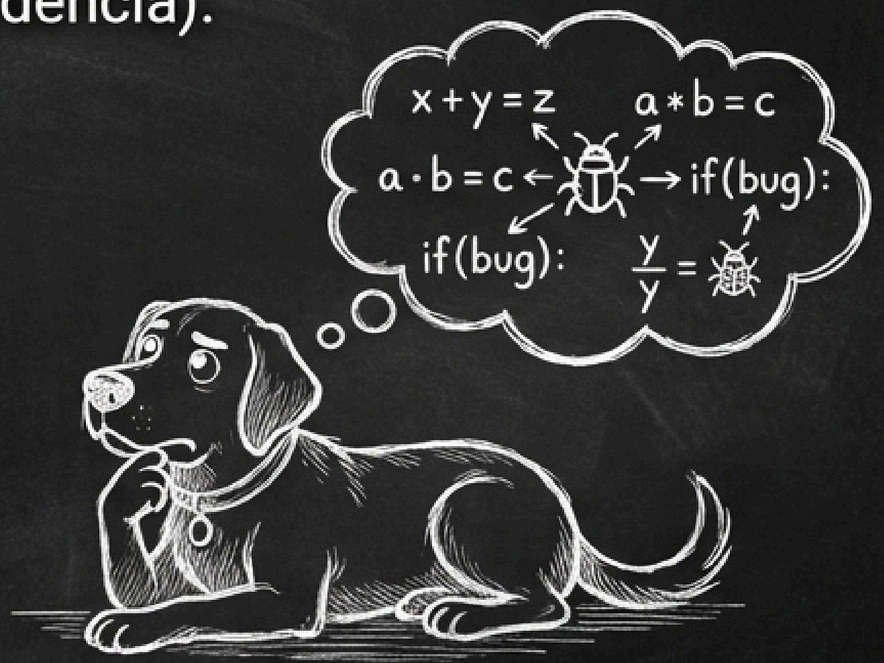
Fases 1 & 2: Isolar o Suspeito e Criar a Teoria

Fase 1: Reproduzir o Bug

Descubra a sequência exata que faz o erro acontecer 100% das vezes. Isole a vítima!

Fase 2: Hipotetizar a Causa

Baseado no Traceback, formule um palpite educado. Evite alterar código aleatoriamente (programar por coincidência).



SANTA CRUZ
CENTRO UNIVERSITÁRIO

Fases 3 & 4: A Intervenção Científica e a Limpeza

Fase 3: Testar a Hipótese

Mude apenas UMA coisa de cada vez. Múltiplas alterações simultâneas contaminam a cena.

Fase 4: Limpar a Cena

Apague breakpoints velhos e remova prints órfãos. Um detetive não deixa bagunça.

O que acontece quando você não testa com cautela.

Táticas de Sobrevivência

O Cérebro Saturado: Seus olhos deixam de enxergar erros de lógica após longas horas de foco.

O Resgate: Respire ar fresco, faça uma caminhada, coma um cookie de chocolate.

Terceirize: Busque no Google, dialogue com IAs (exigindo entender a resposta), mas descanse primeiro. A clareza vem na pausa.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Fase 5: O Veredito Final e os Casos Extremos

SANTA CRUZ
CENTRO UNIVERSITÁRIO

CASO ENCERRADO

O erro original sumiu? Ótimo.
O trabalho acabou? **Ainda não!**

Teste os Casos Extremos (Edge Cases): O que acontece com listas vazias? Textos **em vez de números?** Garanta que o curativo não quebrou o resto do sistema.

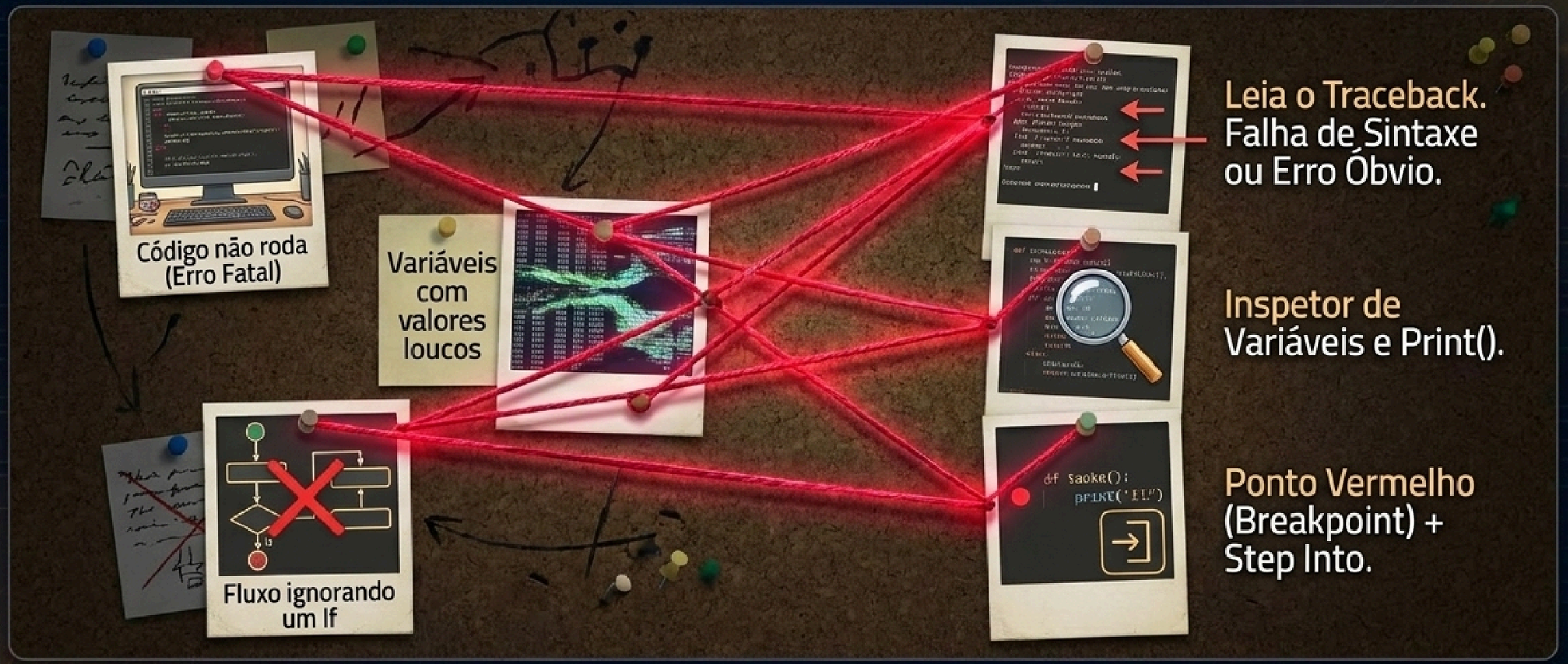
Limpe a Cena do Crime

Checklist de Entrega

- [ ] **Remova absolutamente** todos os **print()** usados durante a investigação.
- [ ] **Apague** breakpoint() soltos pelo terminal.
- [ ] **Teste Casos Extremos (Edge Cases):** Seu conserto funciona para strings vazias? Listas vazias? Teste antes de declarar o caso encerrado.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Quadro de Evidências: Armas Certas para os Inimigos Certos



Resumo Executivo: Não use a ferramenta mais pesada (Debugger) se o interpretador já aponta na linha 1 que falta um : no código.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 02 Aula 07

Caça aos Bugs Lógicos

Parte 01 - Soneca de gato com bugs

Nós escrevemos um programa para calcular o tempo total que o gato dorme em um dia. Mas algo parece estranho... O total que a nossa função retorna não corresponde ao tempo real que o gato passou tirando sonecas.

Objetivo:

* Mesmo que você identifique o erro neste código logo de cara, pratique usar comandos print, um ponto de interrupção (breakpoint) ou o registro de rastreamento (traceback log) para descobrir o que está errado e com que tipo de bug estamos lidando.
* Além disso, corrija o problema para que a função retorne o tempo total de soneca correto.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 02 Aula 07

```
def nap_time(hours_list):
    total = 0
    counter = 1
    while counter < len(hours_list):
        total += hours_list[counter]
        counter += 1
    return total
cat_naps = [2, 3, 1.5, 4]
print("Total nap time:", nap_time(cat_naps))
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 02 Aula 07

Caça aos Bugs Lógicos

Parte 02 - Quanto o seu gato mia?

Imagine que você escreveu este programa para calcular com que frequência o seu gato mia, em média, por hora acordado. Você manteve um registro de quanto ele miou e de quanto ele dormiu a cada dia. Você arredondou os tempos de soneca; afinal, você é um desenvolvedor de software, não um veterinário. Tudo no seu código parece estar perfeitamente bem. Mas adivinha? O programa trava mesmo assim.

Objetivo:

Hora de depurar (debug)! Use o traceback, declarações print e/ou pontos de interrupção (breakpoints) para descobrir o que está causando o travamento e com que tipo de bug você está lidando. Pense em uma maneira inteligente de modificar o código para que ele não trave mais.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 02 Aula 07

```python
def average_meows(logs):
    total_meows_per_hour = 0

    for log in logs:
        nap_hours = 0

        for nap in log["naps"]:
            nap_hours += nap

        awake_hours = 24 - nap_hours
        total_meows_per_hour += log["meows"] / awake_hours

    return total_meows_per_hour / len(logs)

logs = [
    {"day": "Monday", "naps": [1, 2, 5], "meows": 90},
    {"day": "Tuesday", "naps": [4, 3, 2], "meows": 75},
    {"day": "Wednesday", "naps": [8, 6, 9, 1], "meows": 9},
    {"day": "Thursday", "naps": [1, 1, 5, 7], "meows": 38},
    {"day": "Friday", "naps": [3, 3, 2, 8], "meows": 94},
    {"day": "Saturday", "naps": [4, 8, 3], "meows": 64},
    {"day": "Sunday", "naps": [3, 3, 2, 8], "meows": 37}
]

print("Average meows per awake hour:", average_meows(logs))
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 02 Aula 07

Caça aos Bugs Lógicos

Parte 03 - Bugs na Trilha

Você está escrevendo um programa que calcula a distância total de uma trilha. Os dados são flexíveis, e cada segmento da trilha é passado como uma tupla que consiste na distância e na unidade. A função deve imprimir o total de todos esses segmentos. O programa roda sem travar (não quebra), mas algo sobre esse total parece estar errado...

Objetivo:

Use técnicas de depuração (debugging) para descobrir o que há de errado com este código e conserte o bug:

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 02 Aula 07

```python
def total_distance(*segments):
    total = 0
    for length, unit in segments:
        if unit.lower() == "M":
            total += length
        elif unit.lower() == "km":
            total += length / 1000

    print("Total distance:", total, "meters")

total_distance(
    (2.5, "km"),
    (400, "M"),
    (1.0, "km"),
    (800, "m"),
    (1.3, "KM"),
    (900, "m "),
    (1.1, "km")
)
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Tratando Erros

Fonte: Putten e Putten (2026)

Capturando o Erro: A Estrutura Try-Except

A Zona de Risco

Tente executar este código. Se funcionar, o programa ignora o bloco seguinte.

```
try:
    print("Ótimo trabalho," name)
```

A Rede de Segurança

Se houver uma falha estrutural, **aborte o plano inicial** e execute este plano **B** em vez de encerrar o programa.

```
except:
    print("Oh não, algo deu errado.")
```

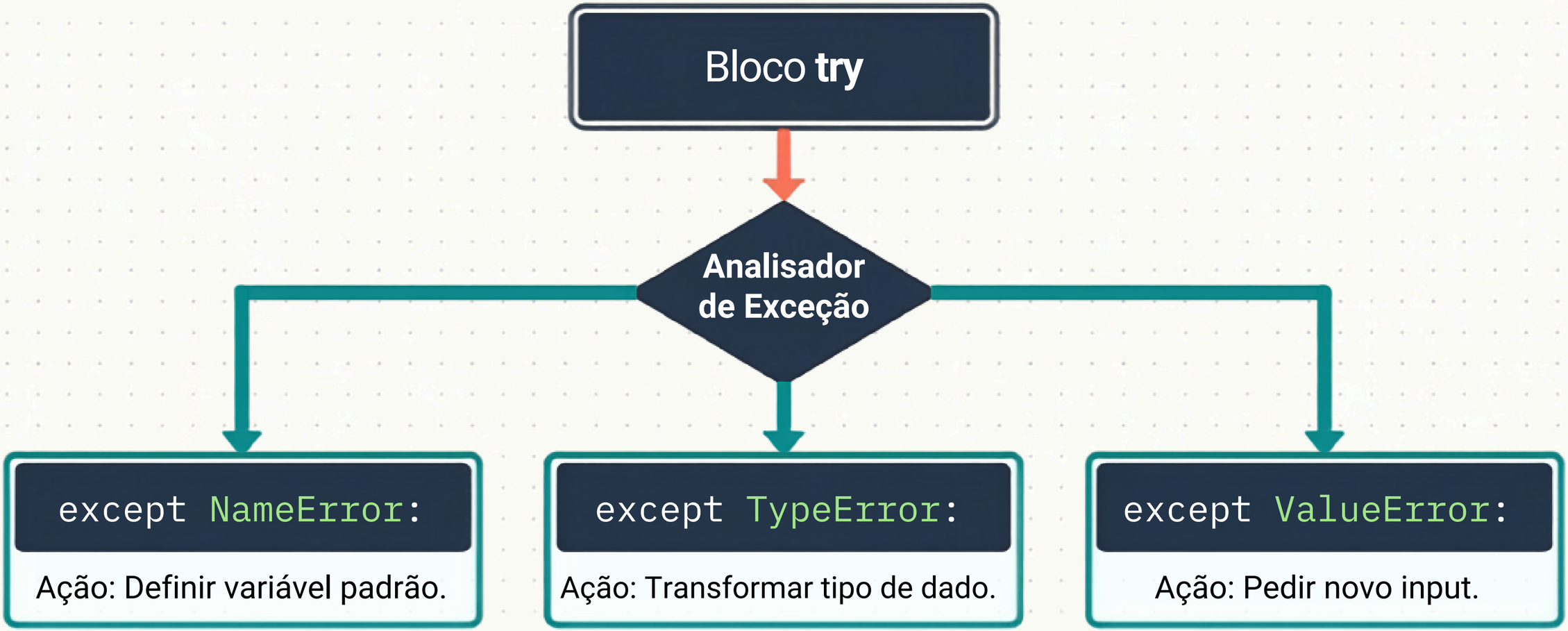
SANTA CRUZ
CENTRO UNIVERSITÁRIO

Prática Tóxica

```
try:
        fazer_algo_arriscado()
except:
      pass
```

Capturar qualquer erro sem especificar **oculta bugs estruturais,** mascara erros lógicos e torna o rastreamento do traceback impossível. Você não conserta o problema; você apenas perde a visibilidade.

Erros: Respondendo com Precisão



Múltiplos blocos atuam como filtros específicos.
O Python direciona o erro para a resposta exata.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

A Rota do Sucesso: O Bloco Else

O 'else' significa 'E então...'. Ele SÓ é executado se o bloco 'try' passar ileso, sem disparar alarmes.

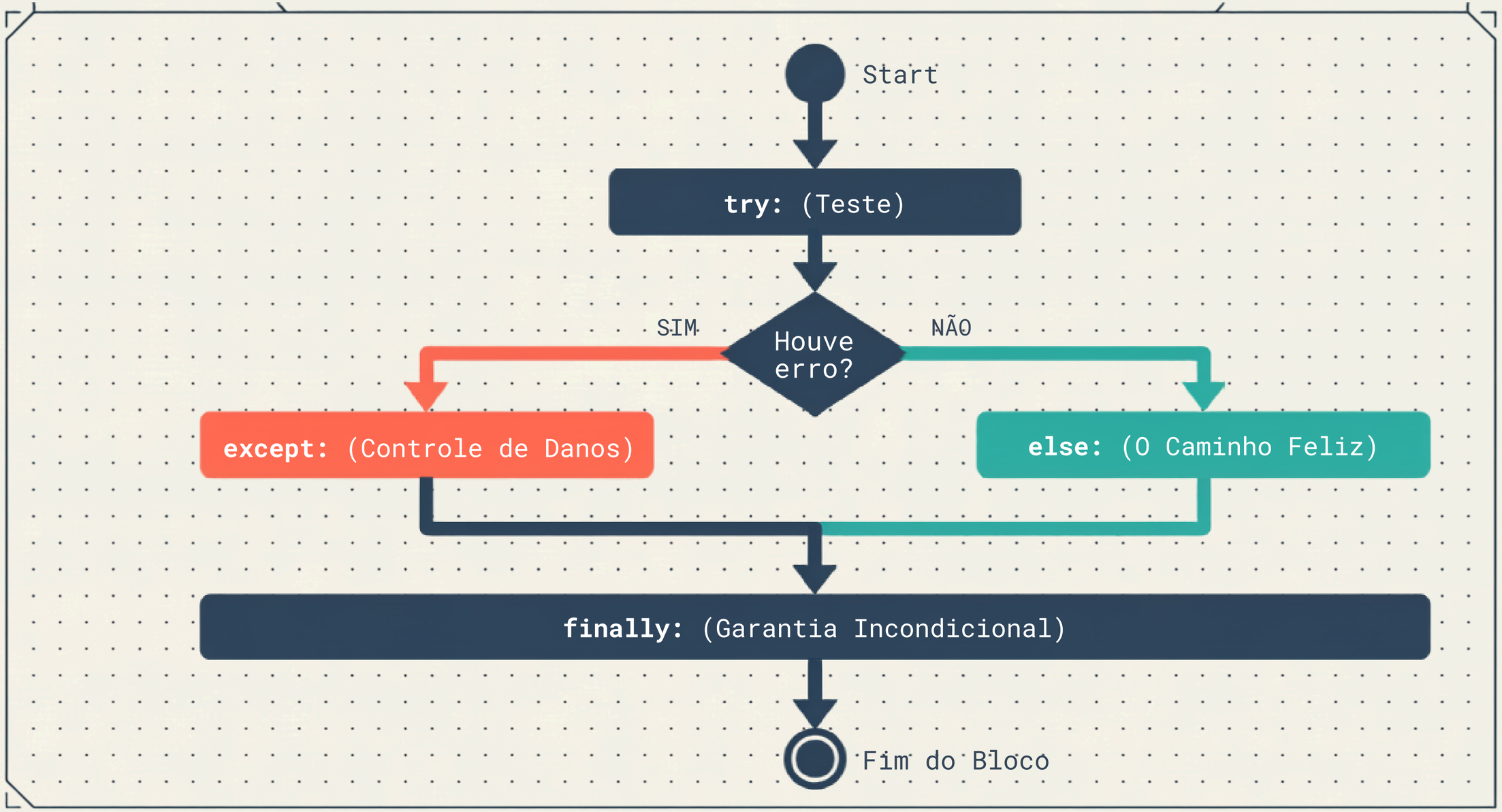
```
try:
    nome = "Batatinha"
    print("Bom trabalho,", nome)
except NameError:
    print("Deu errado.")
else:
    print("Tudo ocorreu perfeitamente!")
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

O Bloco finally

SANTA CRUZ
CENTRO UNIVERSITÁRIO

A Arquitetura Completa de Resiliência



SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 03 Aula 07

Caça aos Bugs e Tratamento de Erros

Você está ajudando um museu local a configurar uma máquina de ingressos. Você escreveu um programa em Python para ela: o visitante digita a sua idade, e a máquina imprime o tipo de ingresso que ele recebe. A lógica é simples:

1. Crianças menores de 12 anos têm acesso gratuito;
2. Adolescentes menores de 18 anos recebem um desconto para jovens;
3. Adultos menores de 65 anos pagam o valor integral;
4. Pessoas com 65 anos ou mais recebem um desconto para idosos;

- Porém você deve tratar o código para que independente doque o visitante digite o programa não dê erro.
- Caso seja uma idade válida o programa deve dar as boas vindas. Caso não deve pedir redigitação.
- E independente do usuário digitar certo ou errado o sistema deve limpar a tela para aguardar pelo próximo visitante.

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Atividade Prática 02 Aula 07

```
def ticket_type():
    age = input("Enter your age: ")
    if age < 12:
        print("You get in for free. Lucky you!")
    elif age < 18:
        print("You get a youth discount!")
    elif age < 65:
        print("That's a full-price ticket for you.")
    else:
        print("You get a senior discount!")

ticket_type()
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Próximos Passos

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Revisão,
Lista de Exercícios,
Prova.

Bugs Não São Fracassos, São Quebra-Cabeças

Cada erro resolvido aprimora seu entendimento do código.
Com as ferramentas e o método certo, um bug deixa
de ser um bloqueio e torna-se um degrau para o conhecimento.

```
Process finished with exit code 0
```

SANTA CRUZ
CENTRO UNIVERSITÁRIO

exit() ou dúvidas?

SANTA CRUZ
CENTRO UNIVERSITÁRIO

OBRIGADA!

“Although practicality beats purity.
Errors should never pass silently.”
The Zen of Python, by Tim Peters

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Referências: PUTTEN, Maaike van; PUTTEN, Imke van. Python Illustrated: Not another boring Python book, learn programming the fun way. Birmingham: Packt Publishing, 2026. [1, 2, 3]

SANTA CRUZ
CENTRO UNIVERSITÁRIO

Para mais informações acesso o **QRCODE** e saiba mais

(41)3052-4900

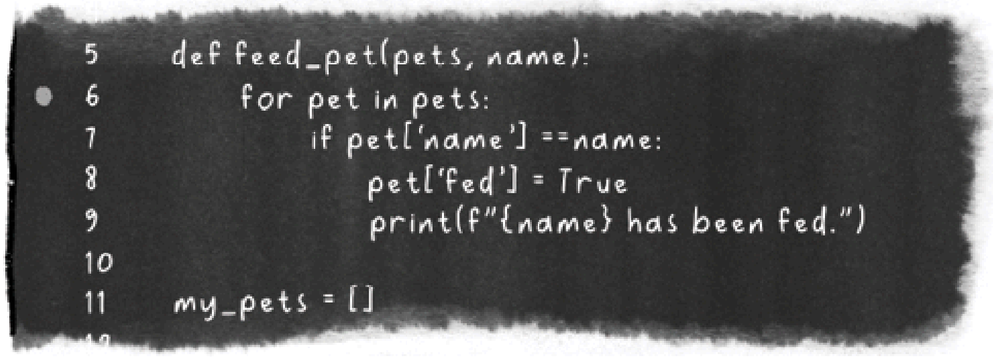
/UniSantaCruzCtba

@unisantacruz

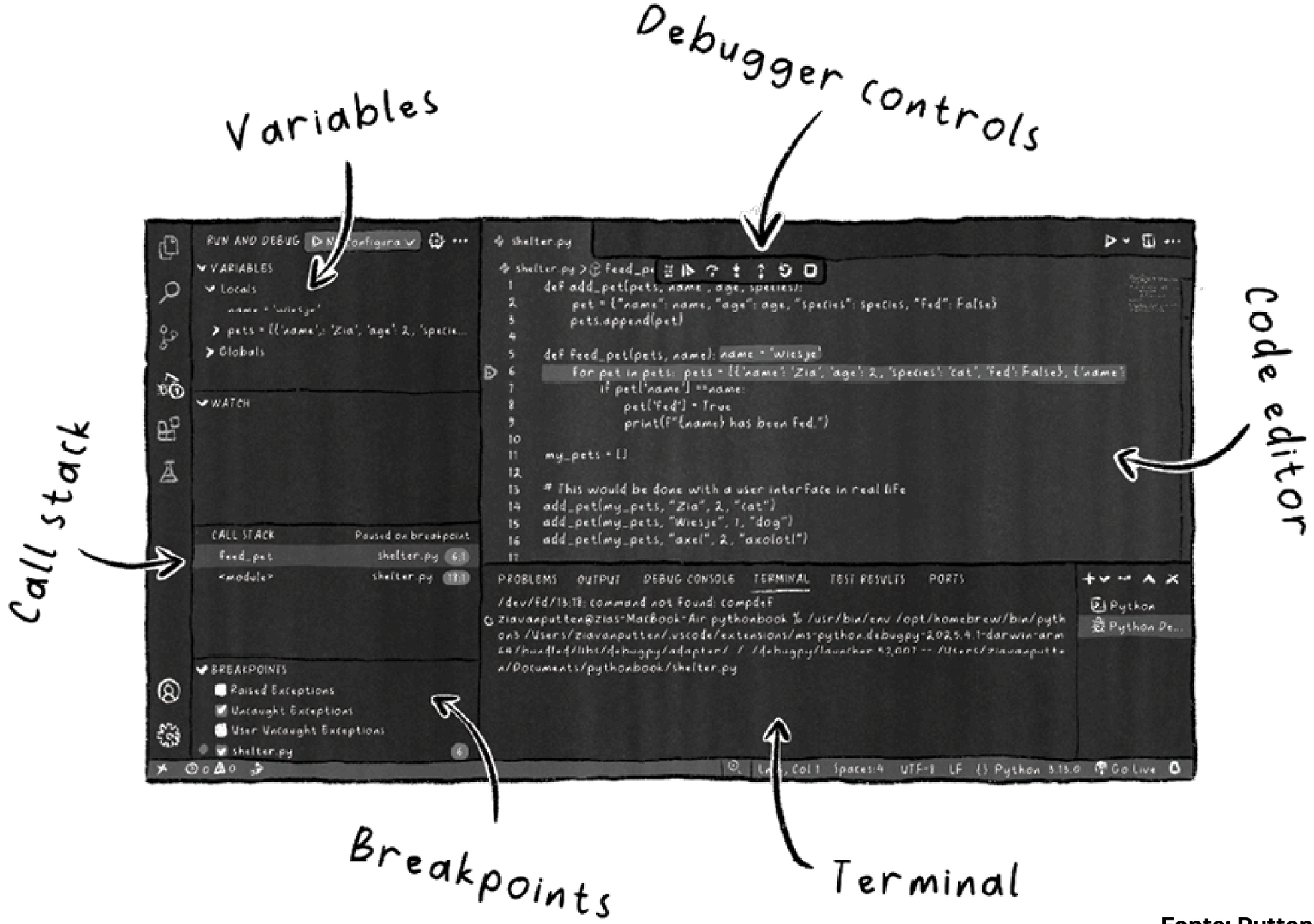
unisantacruztem.com.br

Debug tab

Breakpoint



Fonte: Putten e Putten (2026)



Fonte: Putten e Putten (2026)

tentei debugar o
código,

mas surgiram 5
novos problemas.

Fonte: Putten e Putten (2026)